Βιβλιογραφία

[1] L. B. Wilson and R. G. Clark, Comparative Programming Languages. Pearson Education, 2001.

[2] C. Böhm and G. Jacopini, “Flow diagrams, turing machines and languages with only two formation rules,” Commun. ACM, vol. 9, no. 5, pp. 366–371, May 1966, doi: 10.1145/355592.365646.

[3] “Exception handling,” Wikipedia. Sep. 19, 2021. Accessed: Oct. 15, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Exception_handling&oldid=1045188202

[4] K. Wiegers and J. Beatty, Software Requirements, 3rd edition. Redmond, Washington: Microsoft Press, 2013.

[5] “javadoc-The Java API Documentation Generator.” https://docs.oracle.com/javase/7/docs/technotes/tools/windows/javadoc.html (accessed Sep. 14, 2021).

[6] “How to Write Doc Comments for the Javadoc Tool.” https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html (accessed Sep. 14, 2021).

Κεφάλαιο 9

Σύνοψη
Σε αυτήν την ενότητα αναλύεται σε βάθος η έννοια της αναδρομής. Καταρχάς, αναλύεται η έννοια σε γενικότερο πλαίσιο, στη συνέχεια εξηγείται η λειτουργία της μνήμης stack (στοίβα) και μετά παρουσιάζονται οι προγραμματιστικές αναδρομικές τεχνικές, δηλαδή η άμεση και η αμοιβαία αναδρομή. Τέλος, περιλαμβάνεται συζήτηση για τα πλεονεκτήματα και μειονεκτήματα της αναδρομής σε σχέση με τις μη αναδρομικές προσεγγίσεις.

Προαπαιτούμενη γνώση
Οι θεμελιώδεις τύποι και βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών. Οι δομές επιλογής και οι επαναληπτικές δομές. Η διαχείριση των πινάκων. Οι συναρτήσεις, η έννοια και βασική διαχείριση των απροσδόκητων λαθών, οι προσδιοριστές προσπέλασης και η χρήση των πακέτων.

Λέξεις κλειδιά
Μνήμη stack, αναδρομή, αμοιβαία αναδρομή

9 Αναδρομή

Όταν στον ορισμό μιας έννοιας χρησιμοποιείται η ίδια αυτή έννοια, λέμε πως έχουμε αναδρομικό ορισμό (recursive definition). Χαρακτηριστικό παράδειγμα αναδρομικού ορισμού είναι ο ορισμός της ακολουθίας Fibonacci που είδαμε στην άσκηση 7.8.3. Πιο συγκεκριμένα, αν Fn αναπαριστά τον ν-οστό όρο, η τιμή του υπολογίζεται ως εξής:

$$n = 0 \rightarrow F_n = 0$$
$$n = 1 \rightarrow F_n = 1$$
$$n > 1 \rightarrow F_n = F_{n-1} + F_{n-2}$$

Δηλαδή, αν το n είναι ίσο με το 0 ή το 1, ο ν-οστός όρος ισούται με το n. Για τιμές μεγαλύτερες του 1, ο ν-οστός όρος ισούται με το άθροισμα των δύο προηγούμενων όρων. Επομένως, η τιμή ενός όρου της ακολουθίας Fibonacci ορίζεται με βάση την τιμή δύο όρων της ίδιας ακολουθίας.

Πώς όμως μπορούμε να υπολογίσουμε το F3; Με βάση αυτόν τον ορισμό, F3=F2+F1. Το F1 έχει τιμή 1. Δεν γνωρίζουμε όμως την τιμή του F2. Ωστόσο, γνωρίζουμε πως F2=F1+F0. Επομένως, F2=1 και άρα F3=2.

Ένα άλλο χαρακτηριστικό παράδειγμα είναι ο αναδρομικός ορισμός της δύναμης ενός αριθμού υψωμένου σε εκθέτη. Ας πάρουμε την απλή περίπτωση που η βάση είναι πραγματικός και ο εκθέτης ακέραιος. Σε αυτήν την περίπτωση ο γνωστός μας μη αναδρομικός ορισμός έχει ως εξής;

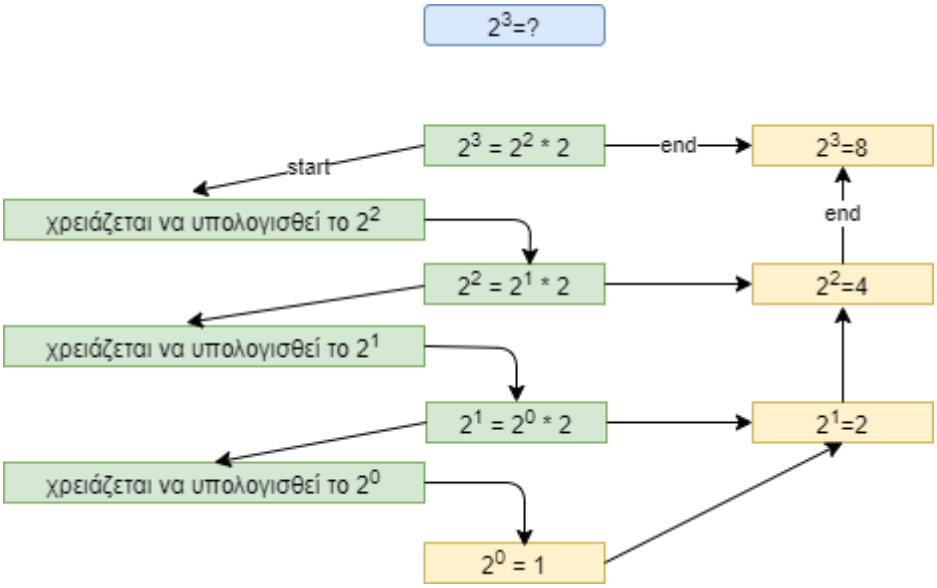
$$a^e = \begin{cases} 1 \; \alpha\nu \; e = 0 \\ a * a * \dots a, e \; \varphi o\rho\acute{\varepsilon}\varsigma \; \alpha\nu \; e > 0 \\ \dfrac{1}{a * a * \dots a, e \; \varphi o\rho\acute{\varepsilon}\varsigma} \; \alpha\nu \; e < 0 \end{cases}$$ [8]

Όμως ισχύει και ο αναδρομικός ορισμός.

$$a^e = \begin{cases} 1, \alpha\nu \; e = 0 \\ a^{e-1} * a, \alpha\nu \; e > 0 \\ \dfrac{1}{a^{-e}}, \alpha\nu \; e < 0 \end{cases}$$

[8] Ας σημειωθεί εδώ πως το 0^0 δεν ορίζεται. Ωστόσο σε πολλές περιπτώσεις θεωρείται πως 0^0=1. Πολλές γλώσσες προγραμματισμού συμπεριλαμβανομένων των Java, Javascript και C, C++, υπολογίζουν το 0^0 ίσο με 1. Για παράδειγμα, αν καλέσετε τη συνάρτηση Math.pow(0, 0), θα διαπιστώσετε πως επιστρέφει 1. Για συμβατότητα θεωρούμε σε αυτό το βιβλίο πως το $0^0 = 1$.

Όπως βλέπουμε στον αναδρομικό ορισμό της δύναμης το a^e ορίζεται ως το $a^{e-1}*a$ όταν το e είναι θετικός και ως $1/a^{-e}$ όταν το e είναι αρνητικός. Πώς θα υπολογίσουμε αναδρομικά την τιμή του 2^3; Ας το συζητήσουμε με τη βοήθεια του σχήματος 9.1.



Σχήμα 9.1 *Ο αναδρομικός υπολογισμός του* 2^3

Όπως φαίνεται στο σχήμα 9.1, για να υπολογίσουμε το 2^3 θα πρέπει πρώτα να υπολογίσουμε το 2^2. Παρόμοια, για να υπολογίσουμε το 2^2, θα πρέπει πρώτα να υπολογίσουμε το 2^1 και για να υπολογίσουμε το 2^1 θα πρέπει να υπολογίσουμε το 2^0. Το 2^0 όμως εξ ορισμού είναι ίσο με το 1. Επομένως τώρα εύκολα υπολογίζουμε το 2^1 που είναι ίσο με 2. Γνωρίζοντας το 2^1, μπορούμε να υπολογίσουμε το 2^2 που είναι ίσο με 4. Τέλος, γνωρίζοντας το 2^2, εύκολα υπολογίζουμε το 2^3 που είναι ίσο με 8.

Γενικότερα, η αναδρομή αποδομεί ένα πρόβλημα τάξης ν σε ένα ή περισσότερα ίδια προβλήματα τάξης μικρότερης του ν. Η διαδικασία της αποδόμησης επαναλαμβάνεται έως ότου φτάσουμε σε ένα ή περισσότερα απλά προβλήματα με προφανή λύση. Τότε τα αποτελέσματα από τα προφανή προβλήματα αξιοποιούνται για τη λύση των προβλημάτων του αμέσως ανώτερου επιπέδου των οποίων η λύση επίσης συνεισφέρει στη λύση των προβλημάτων του ανώτερου επιπέδου, κοκ, έως ότου φθάσουμε στη λύση του αρχικού προβλήματος.

Στη συνέχεια, θα δούμε την αναδρομή στην Java και θα συζητήσουμε τα πλεονεκτήματα και μειονεκτήματά της. Πρώτα όμως θα δώσουμε μερικές απαραίτητες πληροφορίες για μια περιοχή της μνήμης ενός προγράμματος που ονομάζεται στοίβα (stack).

9.1 Η λειτουργία της στοίβας

Για ευκολία θα υποθέσουμε πως κάθε εφαρμογή Java διαθέτει μια μνήμη που ονομάζεται στοίβα[9]. Σύμφωνα με τις προδιαγραφές (specification) της Java [1], η στοίβα μπορεί να είναι σταθερού ή δυναμικά μεταβαλλόμενου μεγέθους. Και στις δύο περιπτώσεις, η στοίβα μπορεί να γεμίσει με δεδομένα. Στην πρώτη περίπτωση, αφού καλυφτεί το προκαθορισμένο μέγεθός της και στη δεύτερη αφού καταλάβει όλη τη διαθέσιμη δυναμική μνήμη. Αν το πρόγραμμά μας απαιτεί επιπλέον μνήμη στοίβας, τότε έχουμε αδυναμία συνέχισης της εκτέλεσης του προγράμματος. Επομένως, η εξάντληση της στοίβας είναι ένα πολύ σοβαρό πρόβλημα που πάντα πρέπει να λαμβάνει ο προγραμματιστής υπόψη του.

Ας δούμε όμως τι αποθηκεύεται στη στοίβα και ποια μπορεί να είναι η πιθανή αιτία εξάντλησής της. Στον κώδικα 9.2 παρουσιάζουμε μια απλή μελέτη περίπτωσης.

```
public class StackFunctionality {
```

[9] Στην πραγματικότητα, στις πολυνηματικές (multithreading) εφαρμογές, διαμορφώνεται μια στοίβα για κάθε νήμα.

```
    static void a() {
        String id = "static function a";
        System.out.println("start " + id);
        String end = "end";
        b();
        System.out.println(end + " " + id);
    }

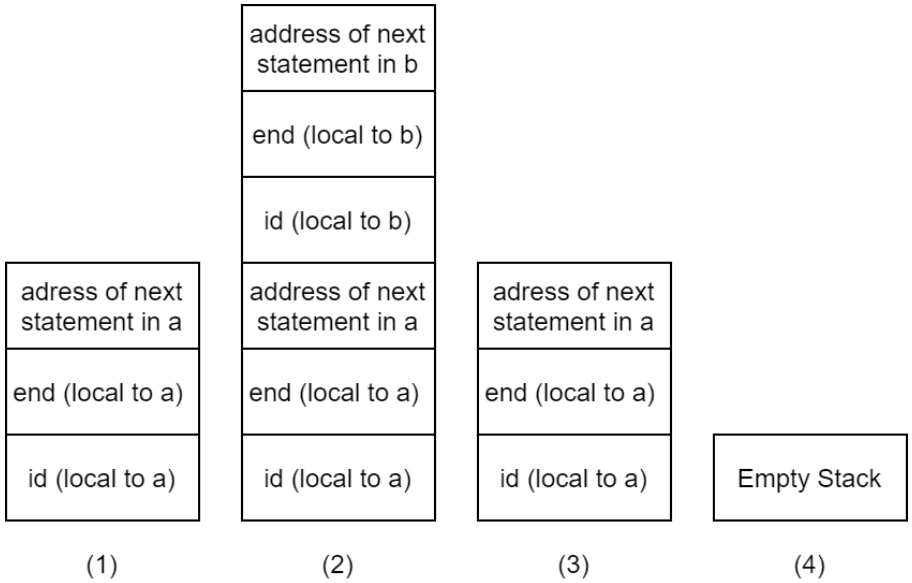
    static void b() {
        String id = "static function b";
        System.out.println("start " + id);
        String end = "end";
        c();
        System.out.println(end + " " + id);
    }

    static void c() {
        String id = "static function c";
        System.out.println("start " + id);
        String end = "end";
        System.out.println("c is running");
        System.out.println(end + " " + id);
    }

    public static void main(String[] args) {
        a();
    }
}
```

Κώδικας 9.1 *Περίπτωση μελέτης λειτουργίας της στοίβας*

Όπως βλέπουμε στον κώδικα 9.1, η συνάρτηση a καλεί την b και αυτή με τη σειρά της, την c. Όταν όμως η c θα ολοκληρωθεί, θα πρέπει η b να συνεχίσει. Από ποιο σημείο όμως; Μα από τη γραμμή της a που έπεται της κλήσης της c. Το ίδιο θα συμβεί όταν ολοκληρωθεί η εκτέλεση της b, δηλαδή θα πρέπει να συνεχίσει η a. Προφανώς, από τη γραμμή του κώδικα που έπεται της κλήσης της b. Η συνάρτηση a προσπελαύνει τις μεταβλητές end και id. Όμως, η b διαθέτει δικές της τέτοιες μεταβλητές. Η προσπέλαση με τα αναγνωριστικά end και id μέσα στην b αναφέρεται στις τοπικές μεταβλητές της b. Αντίθετα, οι τοπικές μεταβλητές της a είναι απροσπέλαστες κατά την εκτέλεση της b. Επομένως, είναι φανερό πως η κλήση συνάρτησης μέσα από άλλη συνάρτηση απαιτεί μια ειδική διαχείριση. Η διαχείριση αυτή βασίζεται στη στοίβα.
Η στοίβα είναι μια Last In First Out (LIFO) δομή, δηλαδή ό,τι μπαίνει τελευταίο στη στοίβα βγαίνει πρώτο ή αλλιώς ό,τι εισάγεται στη στοίβα, τοποθετείται στην κορυφή της και ό,τι εξάγεται, λαμβάνεται από την κορυφή της.



Σχήμα 9.2 *Η στοίβα κατά την εκτέλεση της main του κώδικα 9.1*

Ας δούμε όμως πώς λειτουργεί η στοίβα κατά την εκτέλεση του κώδικα 9.1. Για απλοποίηση, στην περιγραφή αυτή παραλείπουμε τις κλήσεις πέραν των κλήσεων της a, της b και της c. Καταρχάς, έχουμε την κλήση της a. Η a κατανέμει τις τοπικές μεταβλητές της και προχωρά στην εκτέλεσή της μέχρι του σημείου που καλεί την b. Πριν ακριβώς ξεκινήσει η b, οι τοπικές μεταβλητές της a αποθηκεύονται στη στοίβα όπως δείχνει το σχήμα 9.2-1. Όπως φαίνεται σε αυτό το σχήμα, εκτός από τις τοπικές μεταβλητές, στη στοίβα αποθηκεύεται και η διεύθυνση του κώδικα από την οποία θα συνεχιστεί η εκτέλεση της a όταν η b θα έχει τερματίσει. Στη συνέχεια εκτελείται η b μέχρι του σημείου που καλείται η c. Πριν την κλήση της c, οι τοπικές μεταβλητές της b και η διεύθυνση της γραμμής που έπεται της κλήσης της c, επίσης αποθηκεύονται στη στοίβα. Έτσι, η στοίβα διαμορφώνεται όπως δείχνει το σχήμα 9.2-2. Με τον τερματισμό της c, η b εξάγει τις απαραίτητες πληροφορίες από την κορυφή της στοίβας και συνεχίζει την εκτέλεσή της, οπότε η στοίβα διαμορφώνεται όπως δείχνει το σχήμα 9.2-3. Παρόμοια, με τον τερματισμό της b, η a εξάγει από τη στοίβα τις πληροφορίες που την αφορούν και οι οποίες μετά την εξαγωγή των πληροφοριών της b, βρίσκονται στην κορυφή της στοίβας. Έτσι, η a συνεχίζει απρόσκοπτα τη λειτουργία της, ενώ η στοίβα έχει αδειάσει (σχήμα 9.2-4).

Επομένως, κάθε φορά που μια συνάρτηση καλείται από μία άλλη, στη στοίβα τοποθετούνται διάφορες πληροφορίες. Άρα είναι δυνατόν αν έχουμε πολύ μεγάλο βάθος κλήσεων, η στοίβα να μην διαθέτει άλλο χώρο. Ωστόσο, κάτι τέτοιο είναι απίθανο να συμβεί μέσα από μη αναδρομικές κλήσεις. Μπορεί όμως να συμβεί από αναδρομικές κλήσεις. Πόσες φορές θα καλέσει μία συνάρτηση τον εαυτό της, εξαρτάται από το μέγεθος του προβλήματος που επιχειρεί να επιλύσει.

9.2 Αναδρομικές συναρτήσεις

Σύμφωνα με τα όσα έχουμε πει, είναι εύκολο να υποθέσουμε πως μια συνάρτηση είναι αναδρομική όταν καλεί τον εαυτό της. Προσέξτε τη συνάρτηση endless στον κώδικα 9.2.

```
static void endless(int i) { //Κώδικας 9.2
    System.out.println(i);
    endless(i + 1);
}
```

Κώδικας 9.2 *Αναδρομική συνάρτηση χωρίς συνθήκη ελέγχου*

Καταρχάς, είναι φανερό πως πρόκειται για αναδρομική συνάρτηση. Αν καλέσουμε endless(0), τότε η συνάρτηση θα εμφανίσει 0 και στη συνέχεια θα καλέσει endless(1). Η endless(1) θα εμφανίσει 1 και θα καλέσει endless(2). Παρόμοια, η endless(2) θα εμφανίσει 2 και θα καλέσει endless(3). Η αναδρομική κλήση θα συνεχιστεί στην ουσία μέχρις ότου γεμίσει η στοίβα και το πρόγραμμα πέσει.

Επομένως, κάθε αναδρομική συνάρτηση πρέπει να περιλαμβάνει μια συνθήκη ελέγχου της αναδρομικής κλήσης. Με άλλα λόγια, μια αναδρομική συνάρτηση πρέπει να περιλαμβάνει κάποιο κώδικα που κάτω από κάποιες συνθήκες θα τερματίζει την αναδρομική κλήση.

```
static void withEnd(int i) { //Κώδικας 9.3
    System.out.println(i);
    if (i < 3) {
        withEnd(i + 1);
    }
}
```

Κώδικας 9.3 *Απλή αναδρομή με συνθήκη τερματισμού*

Όπως φαίνεται στον κώδικα 9.3, η withEnd είναι επίσης αναδρομική. Ωστόσο, η αναδρομική κλήση γίνεται μόνο εφόσον το i είναι μικρότερο του 3. Για παράδειγμα, αν καλέσουμε withEnd(0), η συνάρτηση θα εμφανίσει 0, στη συνέχεια ελέγχει το i, το βρίσκει μικρότερο του 3 καλεί την endless(1). Με την ίδια διαδικασία, τυπώνει 1, μετά 2, καλεί και withEnd(3), εμφανίζει και το 3 και μετά βρίσκει ότι η έκφραση i<3 είναι ψευδής και σταματά την αναδρομική κλήση. Σε αυτήν την περίπτωση μπορούμε να πούμε ότι η withEnd δεν πέφτει. Ισχύει όμως αυτό γενικευμένα; Η απάντηση είναι όχι. Όπως αναφέραμε και προηγουμένως, εξαρτάται από το μέγεθος του προβλήματος. Για παράδειγμα, αν καλέσουμε withEnd(Integer.MIN_VALUE), θα έχουμε τόσες αναδρομικές κλήσεις ώστε η συνάρτηση θα πέσει πολύ πριν κληθεί με παράμετρο 3.

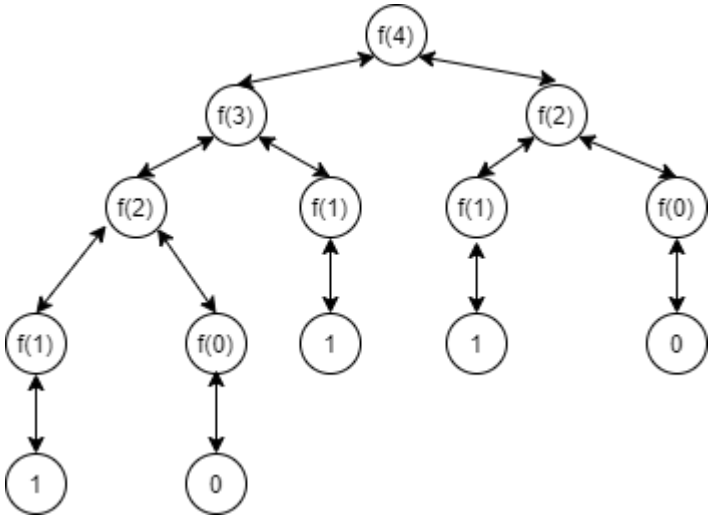
Αυτό είναι και το βασικό μειονέκτημα της αναδρομής. Οποιαδήποτε αναδρομική συνάρτηση μπορεί να αντιμετωπίσει πρόβλημα χωρητικότητας της στοίβας ανάλογα με το μέγεθος του προβλήματος που επιχειρεί να αντιμετωπίσει.

Στον κώδικα 9.4 παραθέτουμε την αναδρομική υλοποίηση υπολογισμού του ν-οστού όρου της ακολουθίας Fibonacci.

```
static int fibonacci(int n) { //Κώδικας 9.4
    if (n < 0) {
        throw new RuntimeException();
    }
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Κώδικας 9.4 *Αναδρομική υλοποίηση υπολογισμού το ν-οστού όρου της ακολουθίας Fibonacci*

Στην ακολουθία Fibonacci δεν ορίζονται όροι αρνητικής τάξης. Έτσι αν η συνάρτηση κληθεί με αρνητικό όρισμα παράγει εξαίρεση. Στη συνέχεια, ο κώδικας 9.4 υλοποιεί ακριβώς τον ορισμό της ακολουθίας που δόθηκε στην αρχή της ενότητας 9. Ας δούμε πώς λειτουργεί η συνάρτηση όταν κληθεί με όρισμα 4. Στο σχήμα 9.3, για απλοποίηση, η συνάρτηση συμβολίζεται με f αντί για fibonacci.



Σχήμα 9.3 *Κλήσεις και τιμές επιστροφής της fibonacci*

Όπως βλέπουμε στο σχήμα 9.3, καταρχάς έχουμε κλήση της f(4). Σύμφωνα με τον κώδικα 9.4, η κλήση αυτή θα επιστρέψει f(3)+f(2). Επομένως, καλείται η f(3) και η f(2) όπως δείχνει και το σχήμα 9.3. Στη συνέχεια, ο υπολογισμός της f(3) απαιτεί την κλήση της f(2) και f(1) και ο υπολογισμός της f(2) απαιτεί την κλήση f(1) και f(0). Σε αυτό το βάθος τρεις κόμβοι του δέντρου, οι f(1), f(1) και f(0), υπολογίζονται χωρίς περαιτέρω αναδρομική κλήση. Αντίθετα ο κόμβος f(2) απαιτεί την κλήση των f(1) και f(0). Κάθε τερματικός κόμβος επιστρέφει την τιμή του στο αμέσως προηγούμενο επίπεδο. Έτσι το f(1) υπολογίζεται ως 1, το f(0) ως 0, το f(2) ως 1, το f(3) ως 2 και το f(4) ως 3.

Ας σημειωθεί πως η περίπτωση κατά την οποία το n είναι μικρότερο ή ίσο του 1 ονομάζεται βασική περίπτωση (base case). Γενικότερα, στην αναδρομή, βασική περίπτωση ονομάζεται η περίπτωση κατά την οποία υπάρχει προφανής λύση που ο υπολογισμός της δεν απαιτεί αναδρομική κλήση.

Το παράδειγμα που θα δούμε τώρα είναι απλό, ωστόσο ιδιαίτερα κρίσιμο για την κατανόηση του μηχανισμού αναδρομικών κλήσεων.

```
static void prt(int i) { // Κώδικας 9.5 prt(1) 123321
    System.out.print(i);
    if (i < 3) {
        prt(i + 1);
    }
    System.out.print(i);
}
```

Κώδικας 9.5 *Συνέχεια της συνάρτησης μετά την αναδρομική κλήση*

Προσέξτε τη συνάρτηση prt του κώδικα 9.5. Τι εμφανίζει κατά την άποψή σας η κλήση prt(3); Συχνά, λαμβάνω λανθασμένη απάντηση σε αυτήν την ερώτηση. Πολλοί φοιτητές στο στάδιο της εκμάθησης της αναδρομής θεωρούν πως η prt(0) θα εμφανίσει 0123. Για να το δούμε βήμα προς βήμα. Όταν αρχίσει η εκτέλεση της prt(0), είναι προφανές πως καταρχάς θα εμφανίσει 0. Στη συνέχεια, ελέγχει τη συνθήκη i<3 την οποία βρίσκει αληθή και εκτελεί το μπλοκ της if. Με την ολοκλήρωση του μπλοκ της if, ο έλεγχος επανέρχεται στην κλήση prt(0). Η συνάρτηση εξάγει την τιμή της i από τη στοίβα και προχωράει στην εκτέλεση της τελευταίας εντολής της στην οποία εμφανίζει και πάλι το 0. Άρα, μέχρι εδώ έχουμε υπολογίσει μια έξοδο ως εξής: 0[Έξοδος από το μπλοκ της if, με i==0]0. Στο μπλοκ της if καλείται καταρχάς, η prt(1) η οποία λειτουργεί ακριβώς με τον ίδιο τρόπο, δηλαδή το αποτέλεσμά της μπορεί να περιγραφεί ως 1[Έξοδος από το μπλοκ της if, με i==1]1 ενώ το συνολικό αποτέλεσμα μέχρι στιγμής είναι 01[Έξοδος από το μπλοκ της if, με i==1]01. Συνεχίζοντας με αυτόν τον τρόπο φτάνουμε στο τελικό αποτέλεσμα που είναι 01233210.

Το κλειδί εδώ είναι να προσέξουμε το εξής: Η prt(0) καλεί κάποια στιγμή την prt(1). Αυτό όμως δεν σημαίνει πως η prt(0) έχει ολοκληρωθεί. Όταν η αναδρομική κλήση θα ολοκληρωθεί, η prt(0) θα συνεχίσει από την επόμενη γραμμή κώδικα από όπου έγινε η κλήση prt(1).

Ας δούμε τώρα την αναδρομική συνάρτηση δύναμης με βάση πραγματικό και εκθέτη ακέραιο.

```
static double power(double b, int e) { //Κώδικας 9.6
    if (e > 0) {
        return power(b, e - 1) * b;
    }
    if (e < 0) {
        return 1 / power(b, -e);
    }
    return 1;
}
```

Κώδικας 9.6 *Αναδρομική υλοποίηση του υπολογισμού δύναμης*

Όταν έχουμε διαθέσιμο έναν αναδρομικό ορισμό, είναι εύκολο να υλοποιήσουμε την αντίστοιχη αναδρομική συνάρτηση. Θυμηθείτε τον αναδρομικό ορισμό της δύναμης στην αρχή της ενότητας 9 και συγκρίνετέ τον με τον κώδικα 9.6. Θα διαπιστώσετε σημαντική ομοιότητα. Στην ουσία, πρόκειται για απλή μεταγραφή από τη γλώσσα των μαθηματικών στη γλώσσα του προγραμματισμού.

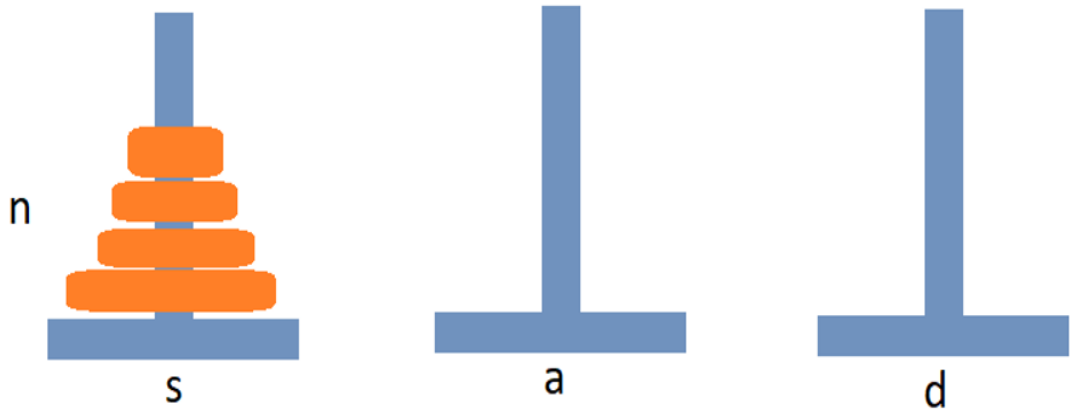
Η συνάρτηση power ελέγχει τον εκθέτη e, αν είναι θετικός επιστρέφει power(b, e-1)*b, δηλαδή ακριβώς ό,τι λέει και ο μαθηματικός ορισμός. Αν πάλι ο εκθέτης είναι αρνητικός, επιστρέφει 1/power(b,-e). Τέλος, εφόσον ο εκθέτης είναι 0, δεν θα γίνουν οι αναδρομικές κλήσεις που προβλέπονται για θετικό και αρνητικό εκθέτη, οπότε η συνάρτηση θα επιστρέψει 1.

9.3 Οι πύργοι του Ανόι

Οι πύργοι του Ανόι (towers of Hanoi) είναι ένα μαθηματικό πρόβλημα που χρησιμοποιείται ευρύτατα για την εκμάθηση της αναδρομής στον προγραμματισμό. Η λύση του φαίνεται αρχικά αρκετά περίπλοκη σε σχέση με τις λύσεις των προβλημάτων που έχουμε δει ως τώρα. Ωστόσο, αν σκεφτεί κανείς με τον κατάλληλο τρόπο, θα διαπιστώσει πως η αναδρομική λύση απαιτεί περιορισμένο κώδικα και τελικά αποδεικνύεται απλούστερη της μη αναδρομικής.

9.3.1 Περιγραφή του προβλήματος

Έχουμε τρεις στύλους, τον s από το source (προέλευση), τον d από το destination (προορισμός) και τον a από το auxiliary (βοηθητικός). Στον στύλο s είναι τοποθετημένοι n δίσκοι όπως φαίνεται στο σχήμα 9.4. Πιο συγκεκριμένα, οι δίσκοι έχουν διαφορετική διάμετρο και είναι τοποθετημένοι έτσι ώστε ένας δίσκος με μεγαλύτερη διάμετρο δεν βρίσκεται ποτέ επάνω από έναν δίσκο με μικρότερη διάμετρο.



Σχήμα 9.4 *Οι πύργοι του Ανόι*

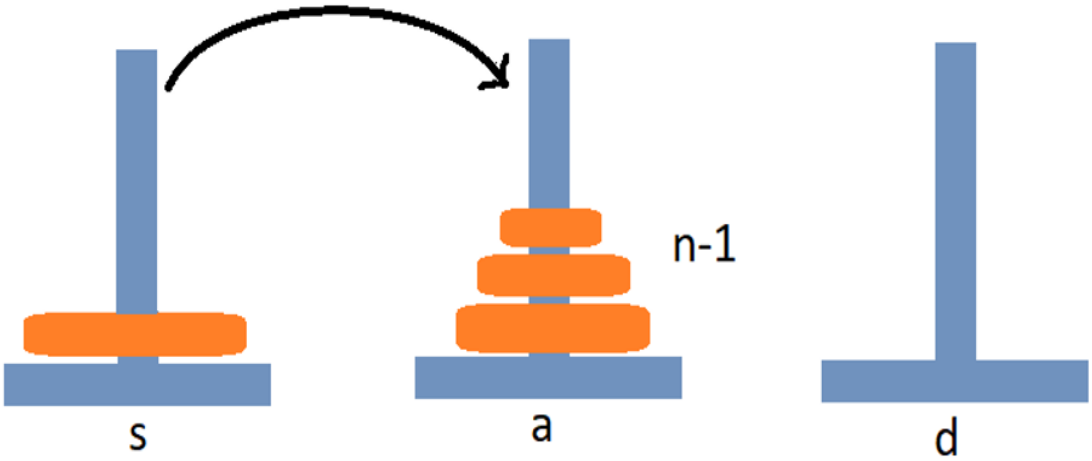
Το ζητούμενο είναι να μεταφέρουμε τους δίσκους από τον στύλο s στον στύλο d χρησιμοποιώντας ως βοηθητικό τον στύλο a. Η μετακίνηση όμως θα πρέπει να γίνει με τους εξής κανόνες:

- Κάθε φορά μπορούμε να μετακινήσουμε έναν μόνο δίσκο από έναν στύλο σε έναν άλλο.

- Δεν πρέπει ποτέ ένας δίσκος με μεγαλύτερη διάμετρο να τοποθετηθεί επάνω από έναν δίσκο με μικρότερη διάμετρο.

9.3.2 Λύση

Η απλούστερη οδός για να λύσουμε το πρόβλημα με αναδρομή είναι να σκεφτούμε με τη φιλοσοφία της αναδρομής. Ας υποθέσουμε πως έχουμε κάποιο τρόπο για να μεταφέρουμε τους n-1 δίσκους από το s στο a χρησιμοποιώντας ως βοηθητικό το d. Σε αυτήν την περίπτωση, στο s θα έχει μείνει ένας δίσκος και στο a θα έχουν τοποθετηθεί οι n-1 δίσκοι όπως δείχνει το σχήμα 9.5. Επομένως, ο δίσκος στο s μπορεί να μετακινηθεί στο d χωρίς πρόβλημα. Αν υποθέσουμε τώρα πως με τον ίδιο τρόπο που μεταφέραμε τους n-1 στύλους στο a, θα τους μεταφέρουμε στο d με βοηθητικό το s, τότε έχουμε λύσει το πρόβλημα. Επομένως, η μεταφορά n δίσκων από το s στο d με βοηθητικό το a έχει αναλυθεί σε δύο προβλήματα μεταφοράς n-1 δίσκων. Αν λύσουμε τα δύο αυτά προβλήματα, έχουμε λύση και στο αρχικό μας πρόβλημα.



Σχήμα 9.5 *Οι n-1 δίσκοι έχουν μετακινηθεί στον στύλο a χρησιμοποιώντας ως βοηθητικό τον στύλο d.*

Πώς θα λύσουμε τώρα αυτά τα δύο προβλήματα; Με τον ίδιο τρόπο. Στο πρώτο πρόβλημα, έχουμε να μεταφέρουμε n-1 δίσκους από το s στο a χρησιμοποιώντας ως βοηθητικό το d. Αν μεταφέρουμε τους n-2 δίσκους από το s στο d χρησιμοποιώντας ως βοηθητικό το a, στον s θα έχουν μείνει δύο στύλοι και ο a θα είναι άδειος, οπότε ο στύλος στην κορυφή του s μπορεί να μετακινηθεί χωρίς πρόβλημα στον a. Στη συνέχεια θα πρέπει να μεταφέρουμε τους n-2 από το d στο a. Επομένως έχουμε αναλύσει το ένα πρόβλημα μεταφοράς n-1 δίσκων σε δύο προβλήματα μεταφοράς n-2 δίσκων.

Η ανάλυση μπορεί να συνεχιστεί για n-1 βήματα. Στο πρώτο βήμα αναλύεται το πρόβλημα μεγέθους n σε δύο υποπροβλήματα μεγέθους n-1, στο δεύτερο βήμα αναλύονται τα δύο υποπροβλήματα μεγέθους n-1 σε τέσσερα υποπροβλήματα μεγέθους n-2. Η διαδικασία συνεχίζεται και στο βήμα n-1 έχει αναλυθεί το πρόβλημα σε 2n-1 υποπροβλήματα μεγέθους n-(n-1) το καθένα, δηλαδή υποπροβλήματα μεγέθους 1. Κάθε πρόβλημα όμως μεγέθους 1, έχει προφανή λύση όπως έχουμε ήδη εξηγήσει. Επομένως, η λύση σε αυτό το επίπεδο, δίνει τη δυνατότητα λύσης στο αμέσως προηγούμενο το οποίο με τη σειρά του δίνει λύση στο προηγούμενό του επίπεδο. Μέσα από αυτήν την αναδρομική διαδικασία φτάνουμε και στη λύση του αρχικού προβλήματος.

```
public class TowersOfAnoi {

    private static void moveOne(int diskNo, char from, char to) {
            System.out.println("move disk " + diskNo + " from " + from + " to " +
to);
    }

    public static void moveDisks(int n, char s, char d, char a) {
        if (n == 1) {
            moveOne(n, s, d);
        } else {
```

```
            moveDisks(n - 1, s, a, d);
            moveOne(n, s, d);
            moveDisks(n - 1, a, d, s);
        }
    }

    public static void main(String[] args) {
        moveDisks(4,'S','D','A');
    }
}
```

Κώδικας 9.7 *Αναδρομική λύση του προβλήματος towers of Hanoi*

Όπως δείχνει ο κώδικας 9.7, έχουμε κωδικοποιήσει το πρόβλημα ως τη συνάρτηση moveDisks που λαμβάνει τέσσερις παραμέτρους. Η παράμετρος n αντιπροσωπεύει τον αριθμό των δίσκων και οι υπόλοιπες τρεις παράμετροι τους ομώνυμους στύλους s, d και a. Με αυτήν την αναπαράσταση μπορούμε να θεωρήσουμε ότι κάθε δίσκος έχει μία ταυτότητα. Ο δίσκος που βρίσκεται πάνω-πάνω στη στοίβα των δίσκων έχει την ταυτότητα 1, ο αμέσως επόμενος την ταυτότητα 2. Έτσι χαρακτηρίζονται όλοι οι δίσκοι μέχρι τον δίσκο στη βάση της στοίβας που είναι ο δίσκος n.

Παράλληλα, έχουμε και τη συνάρτηση moveOne. Πρόκειται για μια απλή συνάρτηση που το μόνο που κάνει είναι να εμφανίζει ένα μήνυμα που μας πληροφορεί ποιος δίσκος μετακινήθηκε και από ποιον στύλο έφυγε και σε ποιον τοποθετήθηκε.

Κατά τα λοιπά, η moveDisks εφαρμόζει ακριβώς τη λογική που περιγράψαμε. Πιο συγκεκριμένα, αν κληθεί με n=1, τότε η μετακίνηση του δίσκου είναι μια απλή μετακίνηση, οπότε καλεί την moveOne. Διαφορετικά, μεταφέρει τους n-1 δίσκους από το s στο a με βοηθητικό το d. Μετά μεταφέρει τον μοναδικό δίσκο στο s και τον τοποθετεί στο d και τέλος μεταφέρει τους n-1 από το a όπου τους έχει τοποθετήσει προηγουμένως στο d χρησιμοποιώντας ως βοηθητικό το s.

9.4 Αμοιβαία αναδρομή

Η αναδρομή κατά την οποία μια συνάρτηση καλεί τον εαυτό της ονομάζεται ειδικότερα άμεση αναδρομή (direct recursion). Είναι όμως πιθανό μια συνάρτηση a να καλεί μια συνάρτηση b και η b να καλεί την a. Πρόκειται και πάλι για αναδρομή που όμως ονομάζεται αμοιβαία αναδρομή (mutual) ή έμμεση (indirect) αναδρομή.

Ας δούμε ένα πρόβλημα που μπορεί να επιλυθεί με αμοιβαία αναδρομή. Έστω ότι έχουμε μια διαδικασία εξέτασης και μια διαδικασία αξιολόγησης της εξέτασης. Η εξέταση βαθμολογεί ένα γραπτό και καλεί την αξιολόγηση. Η δε αξιολόγηση ελέγχει τον βαθμό και αν είναι προβιβάσιμος ενημερώνει με κατάλληλο μήνυμα. Αν όχι, πάλι ενημερώνει με κατάλληλο μήνυμα αλλά καλεί επαναληπτική εξέταση. Η διαδικασία συνεχίζεται μέχρις ότου επιτευχθεί προβιβάσιμος βαθμός.

Ο κώδικας 9.8 δίνει λύση σε αυτό το πρόβλημα με αμοιβαία αναδρομή;

```
    static void exam() { //Κώδικας 9.8
        Random generator = new Random();
        int mark = generator.nextInt(10) + 1;
        evaluate(mark);
    }

    static void evaluate(int mark) { //Κώδικας 9.8
        System.out.print("Your mark is: " + mark);
        if (mark >= 7) {
            System.out.println(" Success");
        } else {
            System.out.println(" Fail");
            exam();
        }
    }
```

Κώδικας 9.8 *Εξέταση και αξιολόγηση με αμοιβαία αναδρομή*

Η συνάρτηση exam παράγει έναν τυχαίο ακέραιο από 1 έως 10. Ο αριθμός αυτός υποτίθεται πως αναπαριστά τον βαθμό ενός γραπτού. Στη συνέχεια καλεί τη συνάρτηση evaluate με παράμετρο τον βαθμό. Η evaluate με τη σειρά της ανακοινώνει τον βαθμό και ανάλογα με την τιμή του ενημερώνει πως έχουμε επιτυχία ή αποτυχία. Στην περίπτωση της αποτυχίας καλεί και πάλι την exam.
Παρόμοια με τη συνάρτηση fibonacci που ορίζεται αναδρομικά, οι συναρτήσεις Hofstadter male και female [2] ορίζονται με αμοιβαία αναδρομή. Πρόκειται για δύο σειρές ακέραιων που δίνονται από τον ακόλουθο ορισμό:

$$\begin{aligned} f(0) &= 1 \\ m(0) &= 0 \\ f(n) &= n - m\big(f(n-1)\big), \qquad n > 0 \\ m(n) &= n - f\big(m(n-1)\big), \qquad n > 0 \end{aligned}$$

Ο κώδικας 9.9 δίνει την υλοποίηση σε Java των συναρτήσεων f και m.

```
static int f(int n) {
    if (n == 0) {
        return 1;
    }
    return n - m(f(n - 1));
}

static int m(int n) {
    if (n == 0) {
        return 0;
    }
    return n - f(m(n - 1));
}
```

Κώδικας 9.9 *Υπολογισμός σειρών Hofstadter male και female*

Παρότι η υλοποίηση των συναρτήσεων με δεδομένο τον αμοιβαία αναδρομικό ορισμό τους είναι σχετικά εύκολη, η παρακολούθηση των κλήσεων και των επιστροφών είναι αρκετά περίπλοκη. Στον πίνακα 9.1, παραθέτουμε τις κλήσεις και επιστροφές που προκαλούνται προκειμένου να υπολογιστούν οι m(2) και f(2).

1	m(2)	f(2)
2	m(1)	f(1)
3	m(0)	f(0)
4	m(0)->0	f(0)->1
5	f(0)	m(1)
6	f(0)->1	m(0)
7	m(1)->0	m(0)->0
8	f(0)	f(0)
9	f(0)->1	f(0)->1
10	m(2)->1	m(1)->0
		f(1)->1
		m(1)
		m(0)
		m(0)->0
		f(0)
		f(0)->1
		m(1)->0
		f(2)->2

Πίνακας 9.2 *Κλήσεις και επιστροφές για τον υπολογισμό των m(2) και f(2)*

Στον πίνακα 9.2 ένα στοιχείο που δεν περιέχει τον χαρακτήρα -> επισημαίνει την είσοδο στην αντίστοιχη συνάρτηση. Αντίθετα τα στοιχεία που περιέχουν -> επισημαίνουν την επιστροφή της συνάρτησης. Ας μελετήσουμε την κλήση m(2).

Προκειμένου να υπολογιστεί η m(2) καλείται η f(m(1)). Κατά την κλήση της f(m(1)) πριν ο κώδικάς της αρχίσει να εκτελείται, θα πρέπει να υπολογιστεί η παράμετρός της, δηλαδή η m(1). Για αυτόν τον λόγο στον πίνακα 9.2 στη δεύτερη γραμμή βλέπουμε κλήση της m(1). Η m(1) υπολογίζεται ως f(m(0)). Κατά την κλήση της f(m(0)), πριν ο κώδικάς της αρχίσει να εκτελείται, θα πρέπει να υπολογιστεί η παράμετρός της, δηλαδή η m(0). Για αυτόν τον λόγο στον πίνακα 9.2 στην τρίτη γραμμή βλέπουμε κλήση της m(0). Η m(0) όμως είναι βασική περίπτωση και υπολογίζεται χωρίς περαιτέρω αναδρομική κλήση. Έτσι στη γραμμή 4 βλέπουμε πως η επιστροφή της m(0) είναι 0. Επομένως, η κλήση f(m(0)) έχει υπολογίσει την παράμετρό της και μπορεί να προχωρήσει στην εκτέλεση του σώματος της συνάρτησης. Έτσι στη γραμμή 5 βλέπουμε την f(0) η οποία επιστρέφει χωρίς περαιτέρω αναδρομική κλήση όπως φαίνεται στη γραμμή 6. Τώρα είναι εφικτός ο υπολογισμός της m(1) όπως φαίνεται στη γραμμή 7. Θυμηθείτε πως η m(2) που είναι το τελικό ζητούμενο είναι ίση με f(m(1)), εφόσον m(1)=0, καλείται η f(0) όπως φαίνεται στη γραμμή 8 που επιστρέφει 1 (γραμμή 9). Τέλος, η m(2) είναι ίση με 1. Παρόμοια, μπορείτε να αναλύσετε την κλήση f(2).

9.5 Πλεονεκτήματα και μειονεκτήματα

Πολλά προβλήματα μπορούν να λυθούν με αναδρομή ευκολότερα με πολύ μικρότερο και πιο κομψό κώδικα από ότι με μη αναδρομικές προσεγγίσεις. Για παράδειγμα, προσπαθήστε να κωδικοποιήσετε τους πύργους του Ανόι μη αναδρομικά και θα διαπιστώσετε πως πρόκειται για πρόβλημα σημαντικής δυσκολίας.

Επιπλέον, πολλά προβλήματα έχουν αναδρομικό ορισμό και επομένως η αναδρομική υλοποίησή τους είναι προφανής.

Από την άλλη πλευρά, η αναδρομή δεν είναι χωρίς μειονεκτήματα. Το σοβαρότερο από όλα είναι η πιθανή εξάντληση της στοίβας. Επομένως, θα πρέπει να χρησιμοποιείτε αναδρομή μόνο εκεί που γνωρίζετε πως τα μεγέθη των προβλημάτων είναι αδύνατο να εξαντλήσουν το μέγεθος της στοίβας. Οι προδιαγραφές της Java επιτρέπουν στοίβα σταθερού μεγέθους ή προσαρμοζόμενου. Εξαρτάται από την υλοποίηση της Java αν υποστηρίζει στοίβα σταθερού ή μεταβλητού μεγέθους. Στην πρώτη περίπτωση, η εξάντληση της στοίβας προκαλεί παραγωγή λάθους τύπου υπερχείλιση της στοίβας (stack overflow) και στη δεύτερη περίπτωση παραγωγή λάθους τύπου υπερχείλιση της μνήμης (memory overflow). Και στις δύο περιπτώσεις, το αποτέλεσμα για την εφαρμογή είναι το ίδιο, δηλαδή ο ανορθόδοξος τερματισμός της.

Επίσης, η αναδρομή σε αρκετές περιπτώσεις μπορεί να δίνει αποτελέσματα σε μεγαλύτερο χρονικό διάστημα από την μη αναδρομική λύση. Αν παρατηρήσετε το δέντρο των κλήσεων για τον υπολογισμό του Fibonacci(4) στο σχήμα 9.3 θα διαπιστώσετε ότι κάποιες κλήσεις επαναλαμβάνονται. Για παράδειγμα, η τιμή f(2) υπολογίζεται 2 φορές. Κατά τον υπολογισμό του εικοστού όρου της ακολουθίας, δηλαδή του f(20), η f(2) καλείται 4181 φορές. Επομένως σε αυτήν την περίπτωση, 4181 φορές επαναλαμβάνεται ο ίδιος υπολογισμός. Ωστόσο αυτό το πρόβλημα έρχεται να λύσει μια ειδική αναδρομική τεχνική γνωστή ως Απομνημόνευση που συζητάμε στην ενότητα 18.

Σε πολλές περιπτώσεις, οι εταιρείες παραγωγής λογισμικού υλοποιούν τη λύση ενός προβλήματος αρχικά αναδρομικά ώστε να βγουν εγκαίρως στην αγορά και στη συνέχεια αντικαθιστούν τη λύση με μη αναδρομική έκδοση. Σε κάθε περίπτωση, μια αναδρομική υλοποίηση μπορεί πάντα να μεταγραφεί με χρήση αποκλειστικά μη αναδρομικών τεχνικών.

Συμπερασματικά, χρησιμοποιήστε αναδρομή όπου σας διευκολύνει και εφόσον σας το επιτρέπει το μέγεθος του προβλήματος που έχετε να λύσετε.

9.6 Λυμένες Ασκήσεις

Σε αυτήν την ενότητα παρουσιάζουμε μια σειρά από παραδείγματα αναδρομικών συναρτήσεων ώστε να βοηθήσουμε τον αναγνώστη να εξοικειωθεί τόσο με την υλοποίηση όσο και με τη χρήση τους.

9.6.1 Παραγοντικό

Να αναπτυχθεί αναδρομική συνάρτηση που επιστρέφει το παραγοντικό της παραμέτρου της.

Λύση
Το παραγοντικό (factorial) ενός ακέραιου n είναι το γινόμενο όλων των θετικών ακέραιων που είναι μικρότεροι ή ίσοι του n. Το παραγοντικό του n συμβολίζεται σαν n!. Επομένως

$$n! = \begin{Bmatrix} 1 \times 2 \times 3 \ldots \times n, για\ n > 0 \\ 1, για\ n = 0 \end{Bmatrix}$$

Εκτός όμως από αυτόν τον ορισμό του παραγοντικού υπάρχει και ένας άλλος, αναδρομικός, ορισμός.

$$n! = \begin{Bmatrix} 1, για\ n = 0\ και\ n = 1 \\ n * (n-1)!, για\ n > 1 \end{Bmatrix}$$

Έχοντας τον αναδρομικό ορισμό είναι εύκολο να υλοποιήσουμε την αντίστοιχη συνάρτηση.

```
    static int factorial(int n) { //Κώδικας 9.10
        if (n == 0 || n == 1) {
            return 1;
        } else {
            return factorial(n - 1) * n;
        }
    }
```

Κώδικας 9.10 *Υλοποίηση υπολογισμού του παραγοντικού με αναδρομή*

9.6.2 Ελάχιστο Κοινό Πολλαπλάσιο

Να αναπτυχθεί συνάρτηση αναδρομικού υπολογισμού του Ελάχιστου Κοινού Πολλαπλάσιου μιας σειράς ακέραιων. Στην main να αναπτυχθεί βασικός κώδικας ελέγχου.

Λύση
Το Ελάχιστο Κοινό Πολλαπλάσιο (ΕΚΠ) μιας σειράς ακέραιων, όπως φανερώνει το όνομά του, είναι το μικρότερο από τα κοινά πολλαπλάσια της σειράς. Υπάρχουν διάφοροι τρόποι υπολογισμού του. Ο τρόπος που χρησιμοποιούμε εδώ βασίζεται στο ακόλουθο σκεπτικό: Ο πρώτος υποψήφιος ακέραιος που μπορεί να είναι το ΕΚΠ της σειράς ακέραιων είναι ο μεγαλύτερος ακέραιος της σειράς. Αν ο ακέραιος αυτός δεν είναι το ΕΚΠ, τότε αναγκαστικά, ΕΚΠ θα είναι κάποιο πολλαπλάσιό του.

```
public class ElaxistoKoinoPollaplasio {

    private static int max(int[] nums) {
        int rVal = nums[0];
        for (int n : nums) {
            if (n > rVal) {
                rVal = n;
            }
        }
        return rVal;
    }

    private static boolean divideAll(int[] nums, int max) {
        for (int n : nums) {
            if (max % n != 0) {
                return false;
            }
        }
        return true;
    }

    public static int eKP(int... nums) { //variable length list of parameters
```

```
        return eKPR(1, nums);
    }

    private static int eKPR(int factor, int... nums) {
        int max = max(nums) * factor;
        if (divideAll(nums, max)) {
            return max;
        } else {
            return eKPR(++factor, nums);
        }

    }

    public static void main(String[] args) {
        System.out.println(eKP(24, 90, 60, 4, 89));
    }
}
```

Κώδικας 9.11 *Αναδρομικός υπολογισμός του ΕΚΠ σειράς ακέραιων*

Στον κώδικα 9.11 η αναδρομική συνάρτηση υπολογισμού του ΕΚΠ είναι η eKPR. Αυτή λαμβάνει ως παραμέτρους τον ακέραιο factor και μια σειρά ακέραιων. Επομένως, η δεύτερη παράμετρός της είναι μεταβλητού μήκους. Στην πρώτη γραμμή της συνάρτησης υπολογίζεται ο μεγαλύτερος ακέραιος της σειράς που είναι ο πρώτος υποψήφιος ακέραιος που πιθανώς είναι το ΕΚΠ. Ο μεγαλύτερος ακέραιος πολλαπλασιάζεται με την παράμετρο factor. Η παράμετρος factor διευκολύνει την αναδρομική κλήση. Κατά την αρχική κλήση της συνάρτησης, η factor πρέπει να έχει την τιμή 1, διαφορετικά ο υπολογισμός του ΕΚΠ πιθανότατα θα καταλήξει σε λάθος. Επομένως, κατά την πρώτη κλήση, η μεταβλητή max έχει την τιμή του μεγαλύτερου ακέραιου της σειράς. Προσέξτε πως η συνάρτηση max δεν επιστρέφει τη θέση του μεγαλύτερου ακεραίου αλλά την τιμή του. Στη συνέχεια ελέγχουμε αν ο max διαιρείται ακριβώς με όλους τους ακέραιους της σειράς. Αυτό επιτυγχάνεται με κλήση της divideAll. Αν η divideAll επιστρέψει true, το ΕΚΠ έχει βρεθεί και επιστρέφεται. Αν όμως επιστρέψει false, ο κώδικας προχωράει σε αναδρομική κλήση αυξάνοντας την factor κατά 1 ώστε να ελεγχθεί το πρώτο πολλαπλάσιο του μεγαλύτερου της σειράς. Στην επόμενη αναδρομική κλήση θα ελεγχθεί το επόμενο πολλαπλάσιο έως ότου βρεθεί ακέραιος που διαιρείται ακριβώς με όλους τους αριθμούς της σειράς εισόδου.

Ίσως να προσέξατε πως όλες οι συναρτήσεις στις οποίες αναφερθήκαμε είναι private, όπως και eKPR. Πράγματι, προκειμένου να κάνουμε την eKPR αναδρομική έχουμε προσθέσει την παράμετρο factor. Ο χρήστης της συνάρτησής μας, όταν θέλει να υπολογίσει το ΕΚΠ μιας σειράς ακέραιων, θα πρέπει να δίνει μόνο την σειρά ακέραιων. Δεν οφείλει να γνωρίζει πως η συνάρτηση είναι αναδρομική και πως κατά την κλήση της πρέπει να της περάσουμε και μια επιπλέον παράμετρο, ούτε τον ενδιαφέρει ο ρόλος αυτής της παραμέτρου. Σε αυτές τις περιπτώσεις κάνουμε την αναδρομική συνάρτηση ιδιωτική και παρέχουμε μια δημόσια συνάρτηση περικάλυμμα (wrapper). Στην περίπτωσή μας η συνάρτηση περιτύλιγμα είναι η eKP η οποία το μόνο που κάνει είναι να καλεί την eKPR με factor ίσο με 1.

9.6.3 Πρώτοι αριθμοί

Να αναπτυχθεί αναδρομική συνάρτηση που εξετάζει αν η παράμετρός της είναι πρώτος αριθμός. Στην main να αναπτυχθεί βασικός κώδικας ελέγχου.

Λύση

Πρώτος είναι ένας ακέραιος που διαιρείται ακριβώς μόνο με τον εαυτό του και το 1. Επιπλέον εξ ορισμού το 1 και το 2 δεν είναι πρώτοι. Επομένως, για να ελέγξουμε αν ένας ακέραιος, n, είναι πρώτος θα πρέπει να ελέγξουμε αν οποιοσδήποτε ακέραιος από το 2 έως n/2 διαιρεί ακριβώς τον n. Αν βρεθεί τέτοιος αριθμός, ο n δεν είναι πρώτος. Είναι προφανές πως κανένας ακέραιος μεγαλύτερος από n/2 δεν μπορεί να διαιρεί ακριβώς τον n.

```
public class PrimeNumber {

    private static boolean isPrime(int num, int divider) {
```

```
        if (num <= 2) {
            return false;
        }
        if (num % divider == 0) {
            return false;
        } else if (divider > num / 2) {
            return true;
        } else {
            return isPrime(num, divider + 1);
        }
    }

    public static boolean isPrime(int num) {
        return isPrime(num, 2);
    }

    public static void main(String[] args) {
        for (int i = 0; i < 100; i++) {
            System.out.println(i + " " + isPrime(i));
        }
    }
}
```

Κώδικας 9.12 *Αναδρομική αξιολόγηση πρώτου αριθμού*

Καταρχάς, στον κώδικα 9.12, στη συνάρτηση isPrime(int, int), ελέγχουμε αν ο υπό εξέταση αριθμός είναι μικρότερος ή ίσος από το 2, οπότε επιστρέφουμε false. Στη συνέχεια προκύπτουν δύο βασικές περιπτώσεις. Στην πρώτη περίπτωση, ο num διαιρείται ακριβώς με τον divider οπότε ο num δεν είναι πρώτος και επιστρέφουμε false. Στη δεύτερη βασική περίπτωση, ελέγχουμε αν ο divider υπερβαίνει το num/2. Σε αυτήν την περίπτωση έχουν ελεγχθεί όλοι οι ακέραιοι από 2 έως num/2 και κανένας δεν βρέθηκε να διαιρεί ακριβώς τον num, επομένως ο num είναι πρώτος και επιστρέφουμε true. Αν καμία από τις βασικές περιπτώσεις δεν προκαλέσει επιστροφή της isPrime, τότε προχωρούμε στην αναδρομική κλήση αυξάνοντας τον divider κατά 1.

Παρόμοια με την προηγούμενη άσκηση, ο divider ως παράμετρος εξυπηρετεί στην αναδρομική κλήση. Ο χρήστης της συνάρτησής μας δεν χρειάζεται να γνωρίζει καν την ύπαρξή του. Έτσι έχουμε κάνει την isPrime(int, int) private και παρέχουμε κατάλληλο περιτύλιγμα, την isPrime(int) που είναι public.

9.6.4 Selection Sort

Να αναπτυχθεί αναδρομική συνάρτηση ταξινόμησης μονοδιάστατου πίνακα ακέραιων με βάση τον αλγόριθμο selection sort (άσκηση 6.5.1). Να κατασκευαστεί κατάλληλος κώδικας ελέγχου με χρήση της Junit.

Λύση

Τη λογική του selection sort έχουμε ήδη παρουσιάσει στην άσκηση 6.5.1. Στον κώδικα 9.13 δίνουμε την αναδρομική και δομημένη υλοποίησή της.

```
import java.util.Arrays;

public class SelectionSort {//Κώδικας 9.13

    private static void swap(int[] array, int idx1, int idx2) {
        int i = array[idx1];
        array[idx1] = array[idx2];
        array[idx2] = i;
    }

    private static int idxOfMin(int[] array, int startIdx) {
        int min = array[startIdx];
        int rVal = startIdx;
        for (int i = startIdx + 1; i < array.length; i++) {
            if (array[i] < min) {
```